DATA CLEANING WALKTHROUGH

Cleaning gold.dim_customers

A beginner-to-intermediate pandas guide, explained step by step

gold_dim_customers.csv • 18,484 rows • 10 columns

337	15	17	0
MISSING COUNTRIES	MISSING GENDERS	MISSING BIRTHDATES	DUPLICATE IDS

Before We Start: Meet the Dataset

Understanding what you're working with is step zero of any cleaning job.

gold.dim_customers is a customer "dimension" table — the kind you'd find in the gold layer of a medallion-architecture warehouse. It has 10 columns describing each customer: keys, names, country, marital status, gender, and two dates.

Column	Type	Example	Issue found
customer_key	int	1	None — clean surrogate key
customer_id	int	11000	None
customer_number	text	AW00011000	None
first_name / last_name	text	Jon / Yang	None
country	text	Australia	**337 missing values**
marital_status	text	Married	None
gender	text	Male	**15 missing values**
birthdate	text (should be date)	1971-10-06	**Stored as text; 17 missing**
create_date	text (should be date)	2025-10-06	**Stored as text**

18,484
Total rows loaded

0
Duplicate customer_id rows

369
Total missing values to fix (across 3 columns)

> **Beginner tip:** Always inspect a dataset before touching it. Two lines — `df.info()` and `df.isnull().sum()` — tell you 80% of what needs cleaning.

---

1 Load the Data & Take a First Look

Never clean blind — always inspect first.

clean_dim_customers.py

```python
import pandas as pd

df = pd.read_csv("gold_dim_customers.csv")
print(df.shape)          # (rows, columns)
df.info()              # column types + non-null counts
df.isnull().sum()    # how many missing values per column
```

What's happening here:

- `pd.read_csv()` reads the CSV file into a **DataFrame** — pandas' table object.
- `.shape` is a quick sanity check: does the row/column count match what you expect?
- `.info()` shows each column's data type. Notice `birthdate` and `create_date` show up as `object` (text) instead of dates — that's a red flag.
- `.isnull().sum()` counts missing (`NaN`) values column by column.

OUTPUT (ISNULL().SUM())

```
country      337
gender       15
birthdate    17
(all other columns: 0 missing)
```

2 Fix Data Types

Dates stored as text can't be sorted, filtered, or used in date math.

> **Why it matters:** as text, `"1971-10-06"` compares alphabetically, not chronologically. Converting to a real datetime unlocks age calculations, date filtering, and correct sorting.

clean_dim_customers.py

```python
df["birthdate"] = pd.to_datetime(df["birthdate"], errors="coerce")
df["create_date"] = pd.to_datetime(df["create_date"], errors="coerce")
```

What's happening here:

- `pd.to_datetime()` converts a text column into pandas' native datetime type.
- `errors="coerce"` is the important beginner trick: instead of crashing on a badly formatted date, pandas turns it into `NaT` ("Not a Time") so you can find and handle it later.

---

3 Handle Missing Values

337 missing countries, 15 missing genders, 17 missing birthdates — three different fixes.

clean_dim_customers.py

```
# Categorical text columns: fill with an explicit label
df["country"] = df["country"].fillna("Unknown")
df["gender"]  = df["gender"].fillna("Unknown")

# Birthdate: don't guess a fake date — flag it instead
df["birthdate_missing"] = df["birthdate"].isna()
```

What's happening here, and why each column gets a *different* treatment:

- **country / gender** — these are categories. Filling missing values with `"Unknown"` keeps the row usable in group-by reports without inventing a fake country or gender.
- **birthdate** — there's no safe "average" or placeholder date. Instead, we add a boolean flag column (`birthdate_missing`) so downstream analysis can filter these rows out of age-related calculations rather than corrupting them.

> **Intermediate tip:** Filling everything with the same value (e.g. 0 or "N/A") is a common beginner mistake. Always ask: *does a default value make sense for this specific column?* If not, flag it instead of faking it.

4 Standardize Text Formatting

Inconsistent capitalization or stray spaces silently break filters and joins.

clean_dim_customers.py

```
text_cols = ["first_name", "last_name", "country",
             "marital_status", "gender"]

for col in text_cols:
    df[col] = df[col].str.strip().str.title()
```

What's happening here:

- `.str.strip()` removes leading/trailing whitespace (e.g. `" Male"` → `"Male"`).
- `.str.title()` normalizes capitalization so `"AUSTRALIA"` and `"australia"` both become `"Australia"` — this prevents them being counted as two different groups.
- Looping over a list of column names avoids repeating the same code five times — a small step toward writing cleaner, more maintainable pipelines.

---

5 Remove Duplicate Records

This dataset has zero duplicates — but always check, and always write the code defensively.

clean_dim_customers.py

```
before = len(df)
df = df.drop_duplicates(subset="customer_id", keep="first")
print(f"Removed {before - len(df)} duplicate rows")
```

What's happening here:

- `subset="customer_id"` tells pandas to treat rows as duplicates when the **business key** repeats — even if some other column differs. This is more reliable than checking every column.
- `keep="first"` keeps the earliest occurrence and drops later repeats.
- Verified on this dataset: **0 duplicate customer_id values** out of 18,484 rows.

6 Validate Business Rules

Turn birthdates into ages, and catch anything impossible.

clean_dim_customers.py

```
import numpy as np

today = pd.Timestamp.today()
df["age"] = ((today - df["birthdate"]).dt.days / 365.25).round(0)
df.loc[df["age"] > 100, "age"] = np.nan  # impossible ages → missing
```

What's happening here:

- Subtracting two datetime columns gives a `Timedelta` ; `.dt.days` pulls out the number of days, which we convert to years.
- `df.loc[condition, column] = value` is the standard pandas pattern for conditional updates — here it wipes out any age over 100, treating it as a data-entry error rather than a real person.
- Dataset check confirmed birthdates range from **1916 to 1986**, all realistic — this rule is a safety net for future data loads.

> **Beginner tip:** Validation rules like this don't need to fire today to be worth writing. They protect the pipeline the next time new (messier) data arrives.

---

7 Save the Clean Dataset

The last step — always confirm the shape changed the way you expect.

clean_dim_customers.py

```
df.to_csv("gold_dim_customers_clean.csv", index=False)
print("Final shape:", df.shape)
```

- `index=False` stops pandas from writing its internal row-number index as an extra column in the CSV.
- Comparing the shape before and after is a cheap final sanity check that nothing was accidentally dropped.

Summary

Step	Action	Result
1	Load & inspect	18,484 rows, 10 columns confirmed
2	Fix data types	birthdate & create_date → real dates
3	Handle missing values	337 countries, 15 genders filled; 17 birthdates flagged
4	Standardize text	Consistent capitalization, no stray spaces
5	Remove duplicates	0 duplicates found (verified on customer_id)
6	Validate business rules	Age column added; impossible ages guarded against
7	Save output	gold_dim_customers_clean.csv written